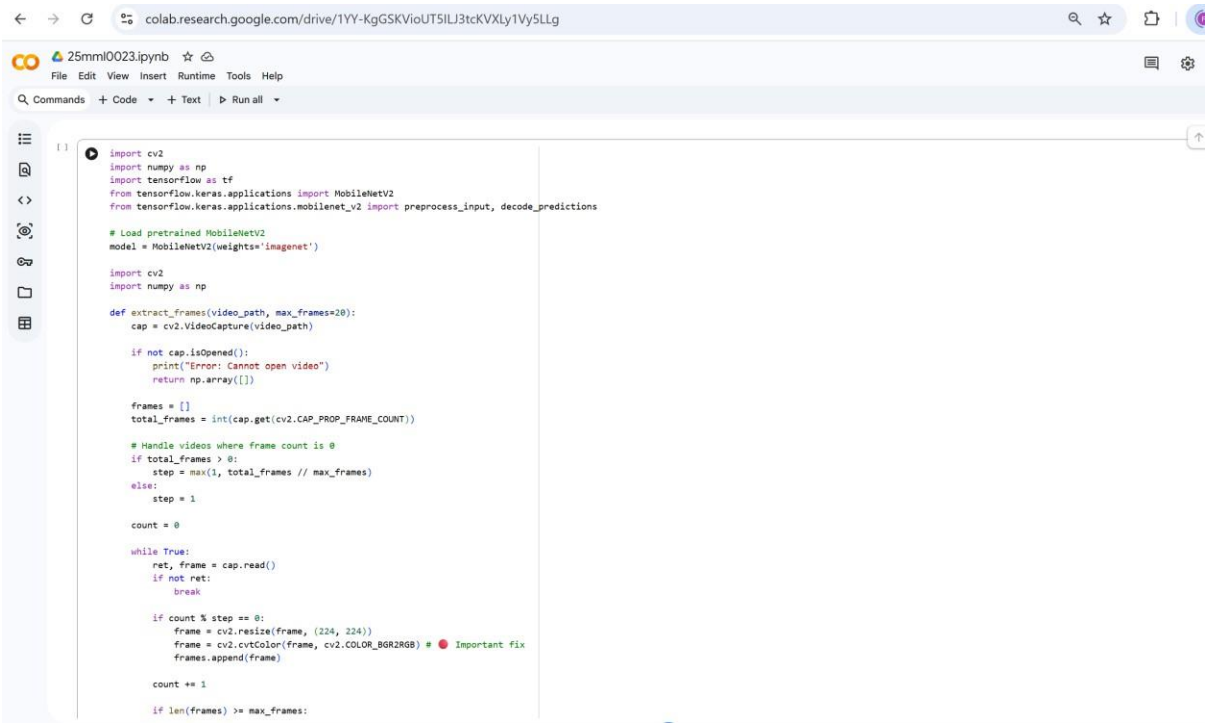
EDGE INTELLIGENCE

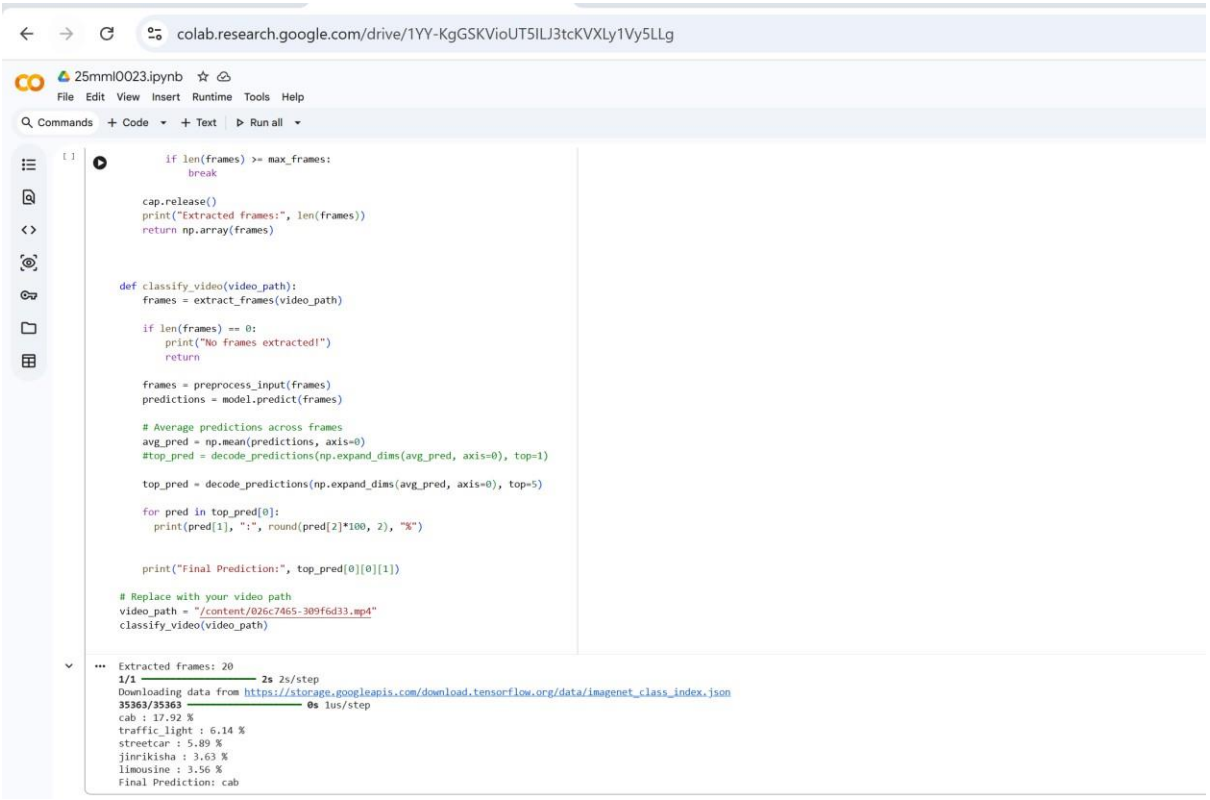
LAB TASK - 3

PRADHEESH RAJ R

25MML0023

CODE-1:





STEPS IN THE CODE:

1. Import required libraries
   Import cv2, numpy, tensorflow, and MobileNetV2 related functions.
2. Load Pretrained Model
   Load MobileNetV2 with weights='imagenet' for image classification.
3. Open Video File
   Use cv2.VideoCapture(video_path) to read the video.
4. Check Video Open Status
   Use cap.isOpened() to ensure the video loads correctly.
5. Calculate Frame Step Size
   Compute step = total_frames // max_frames to sample frames evenly.
6. Read Frames Sequentially
   Use cap.read() inside a loop to extract frames.
7. Resize Frames
   Resize each selected frame to (224, 224) as required by MobileNetV2.
8. Convert Color Format
   Convert BGR → RGB using cv2.cvtColor().

9. Preprocess Frames
   Apply preprocess_input() before feeding into the model.
10. Predict & Average Results
    Predict each frame → Take mean of predictions → Decode top 5 classes → Print final prediction.

DRAWBACKS OF THIS CODE:

1. Uses ImageNet Model for Video
   MobileNetV2 is trained for image classification, not video classification.
2. No Temporal Information
   Frame averaging ignores motion and sequence information.
3. Simple Frame Sampling
   May miss important action frames.
4. No Face/Object Detection Preprocessing
   Entire frame is classified, not specific objects.
5. Limited to 20 Frames
   Important details might be lost.
6. Not Suitable for Deepfake Detection
   ImageNet classes cannot detect fake videos.
7. High Memory Usage for Large Videos
8. No Confidence Threshold Check
9. No GPU Optimization Handling
10. No Error Logging Mechanism

ERRORS IN CODE:

1. Hardcoded Video Path
2. video_path = "/content/026c7465-309f6d33.mp4"

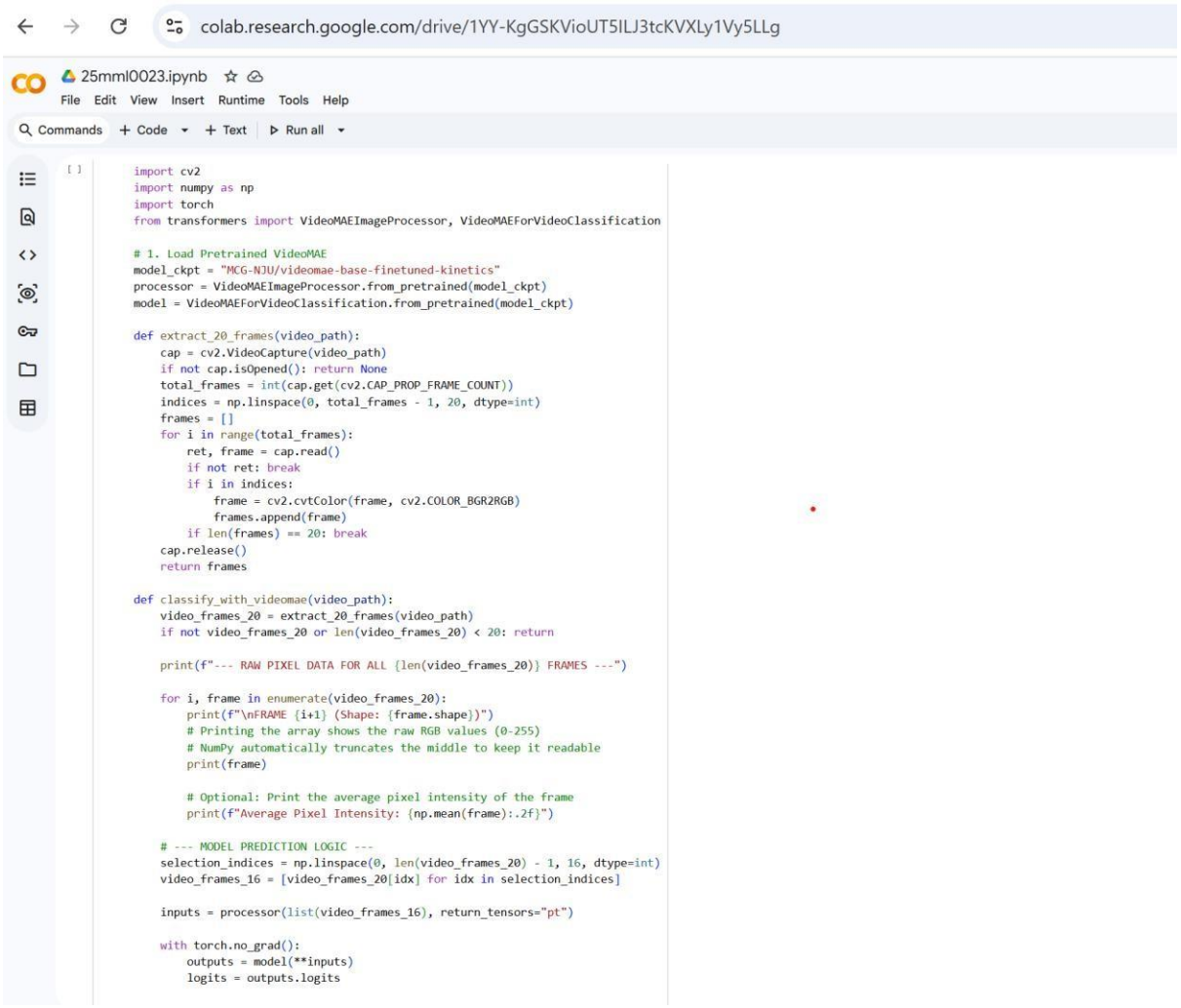
   Should be dynamic input.
3. No Batch Prediction Handling
   For long videos, memory can crash.

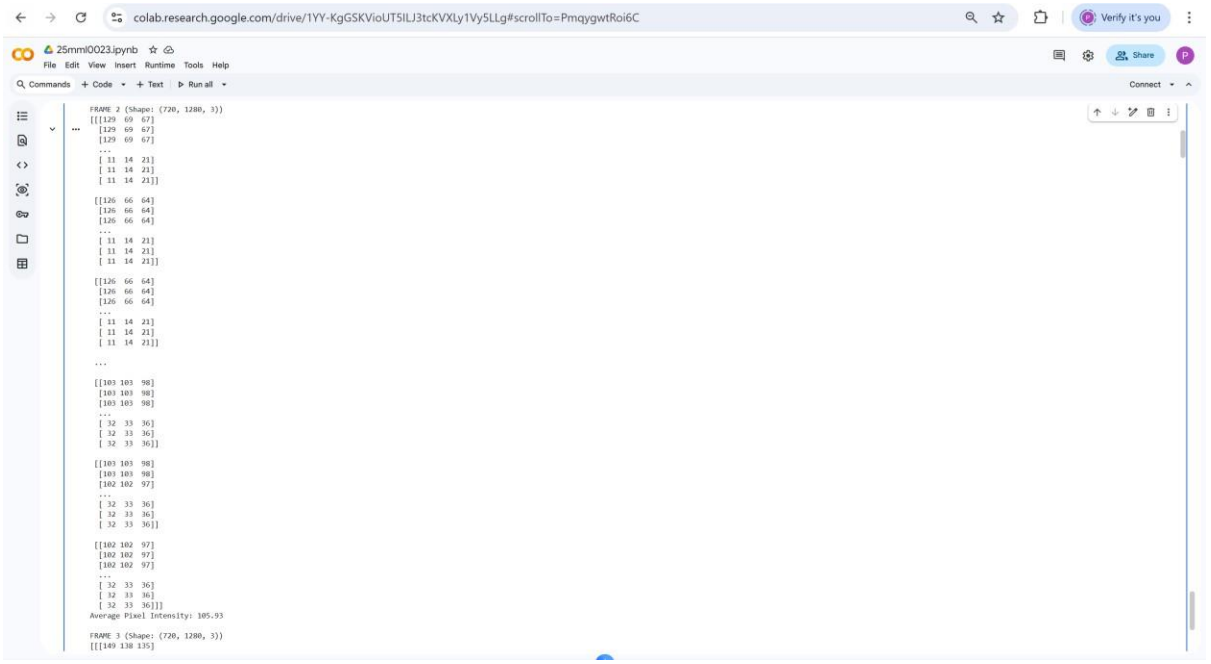
4. No Exception Handling
   Use try-except for safer execution.
5. No Model Summary Printed
   Helpful for debugging.
6. No FPS-Based Frame Sampling
   Better to sample based on time instead of frame count.
7. decode_predictions Used Without Class Mapping Validation
8. No Softmax Check
   Ensure predictions are properly normalized.
9. No Logging for Debugging
10. Not Designed for Custom Dataset
    Only works with ImageNet classes.

BETTER APPROACH (Video Classification):

Instead of MobileNetV2, use:

- 3D CNN (Conv3D)
- LSTM + CNN
- I3D (Inflated 3D ConvNet)
- TimeSformer
- SlowFast Network


colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXLy1Vy5LLg
25mml0023.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
import cv2
import numpy as np
import torch
from transformers import VideoMAEImageProcessor, VideoMAEForVideoClassification
# 1. Load Pretrained VideoMAE
model_ckpt = "MCG-NJU/videomae-base-finetuned-kinetics"
processor = VideoMAEImageProcessor.from_pretrained(model_ckpt)
model = VideoMAEForVideoClassification.from_pretrained(model_ckpt)
def extract_20_frames(video_path):
cap = cv2.VideoCapture(video_path)
if not cap.isOpened(): return None
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
indices = np.linspace(0, total_frames - 1, 20, dtype=int)
frames = []
for i in range(total_frames):
ret, frame = cap.read()
if not ret: break
if i in indices:
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
frames.append(frame)
if len(frames) == 20: break
cap.release()
return frames
def classify_with_videomae(video_path):
video_frames_20 = extract_20_frames(video_path)
if not video_frames_20 or len(video_frames_20) < 20: return
print(f"--- RAW PIXEL DATA FOR ALL {len(video_frames_20)} FRAMES ---")
for i, frame in enumerate(video_frames_20):
print(f"\nFRAME {i+1} (Shape: {frame.shape})")
# Printing the array shows the raw RGB values (0-255)
# NumPy automatically truncates the middle to keep it readable
print(frame)
# Optional: Print the average pixel intensity of the frame
print(f"Average Pixel Intensity: {np.mean(frame):.2f}")
# --- MODEL PREDICTION LOGIC ---
selection_indices = np.linspace(0, len(video_frames_20) - 1, 16, dtype=int)
video_frames_16 = [video_frames_20[idx] for idx in selection_indices]
inputs = processor(list(video_frames_16), return_tensors="pt")
with torch.no_grad():
outputs = model(**inputs)
logits = outputs.logits


colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXLy1Vy5LLg#scrollTo=PmqygwtRoi6C
Verify it's you
25mml0023.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
Connect
FRAME 2 (Shape: (720, 1280, 3))
[[[129 69 67]
[129 69 67]
[129 69 67]
...
[11 14 21]
[11 14 21]
[11 14 21]]
[[126 66 64]
[126 66 64]
[126 66 64]
...
[11 14 21]
[11 14 21]
[11 14 21]]
[[126 66 64]
[126 66 64]
[126 66 64]
...
[11 14 21]
[11 14 21]
[11 14 21]]
...
[[103 103 98]
[103 103 98]
[103 103 98]
...
[32 33 36]
[32 33 36]
[32 33 36]]
[[103 103 98]
[103 103 98]
[102 102 97]
...
[32 33 36]
[32 33 36]
[32 33 36]]
[[102 102 97]
[102 102 97]
[102 102 97]
...
[32 33 36]
[32 33 36]
[32 33 36]]]
Average Pixel Intensity: 105.93
FRAME 3 (Shape: (720, 1280, 3))
[[[149 138 135]

colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXLy1Vy5LLg#scrollTo=PmqygwtRoi6C

25mml0023.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
import cv2
import numpy as np
import torch
import matplotlib.pyplot as plt
from transformers import VideoMAEImageProcessor, VideoMAEForVideoClassification

# 1. Load Pretrained VideoMAE
model_ckpt = "MCG-NJU/videomae-base-finetuned-kinetics"
processor = VideoMAEImageProcessor.from_pretrained(model_ckpt)
model = VideoMAEForVideoClassification.from_pretrained(model_ckpt)

def extract_20_frames(video_path):
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened(): return None

    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    # Step 1: Extract 20 frames evenly from the video
    indices = np.linspace(0, total_frames - 1, 20, dtype=int)

    frames = []
    for i in range(total_frames):
        ret, frame = cap.read()
        if not ret: break
        if i in indices:
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)
        if len(frames) == 20: break
    cap.release()
    return frames

def display_frames(frames):
    """Plots all 20 extracted frames in a 5x4 grid."""
    fig, axes = plt.subplots(5, 4, figsize=(12, 15))
    for i, ax in enumerate(axes.flat):
        if i < len(frames):
            ax.imshow(frames[i])
            ax.set_title(f"Frame {i+1}")
        ax.axis('off')
    plt.tight_layout()
    plt.show()

def classify_with_videomae(video_path):
    # Get your 20 frames
    video_frames_20 = extract_20_frames(video_path)
    if not video_frames_20 or len(video_frames_20) < 20:
        print("Error: Could not extract 20 frames.")
        return

    # Visualise the 20 frames
    display_frames(video_frames_20)
```

colab.research.google.com/drive/1YY-KgGSKVioUT5ILJ3tcKVXLy1Vy5LLg#scrollTo=PmqygwtRoi6C

25mml0023.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
    # Visualise the 20 frames
    display_frames(video_frames_20)

    # Step 2: Select 16 frames from the 20 to satisfy the model's requirements
    # This takes 16 evenly spaced frames from your 20-frame list
    selection_indices = np.linspace(0, len(video_frames_20) - 1, 16, dtype=int)
    video_frames_16 = [video_frames_20[i] for i in selection_indices]

    # Step 3: Preprocess and Predict
    inputs = processor(list(video_frames_16), return_tensors="pt")

    with torch.no_grad():
        outputs = model(**inputs)
        logits = outputs.logits

    probs = torch.nn.functional.softmax(logits, dim=-1)
    top_results = torch.topk(probs, 5)

    print(f"\n--- Top 5 Predictions (Using 20-frame context) ---")
    for i in range(5):
        score = top_results.values[0][i].item()
        label_id = top_results.indices[0][i].item()
        label = model.config.id2label[label_id]
        print(f"{label}: {round(score * 100, 2)}%")

# Usage
video_path = "/content/026c7465-309f6d33.mp4"
classify_with_videomae(video_path)
```

Loading weights: 100% 186/186 [00:00<00:00, 563.74it/s, Materializing param=videomae.encoder.layer.11.output.dense.weight]

colab.research.google.com/drive/1YY-KgGSKVioUT5lLJ3tcKVXLy1Vy5LLg#scrollTo=PmqygwtRoi6C

25mml0023.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

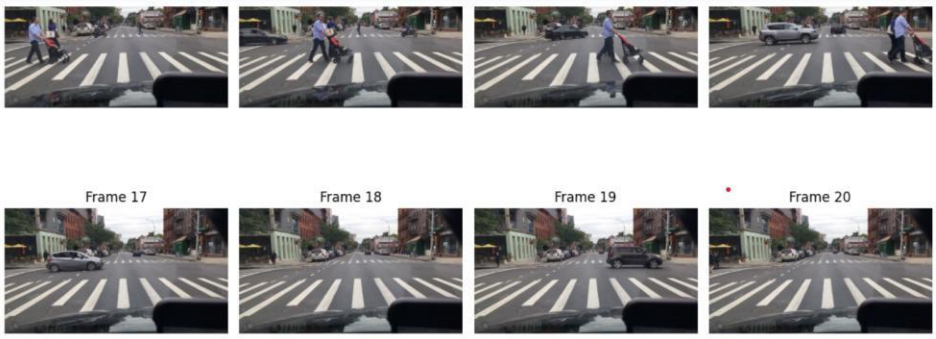
```
# Usage
video_path = "/content/026c7465-309f6d33.mp4"
classify_with_videomae(video_path)
```

colab.research.google.com/drive/1YY-KgGSKVioUT5lLJ3tcKVXLy1Vy5LLg#scrollTo=PmqygwtRoi6C

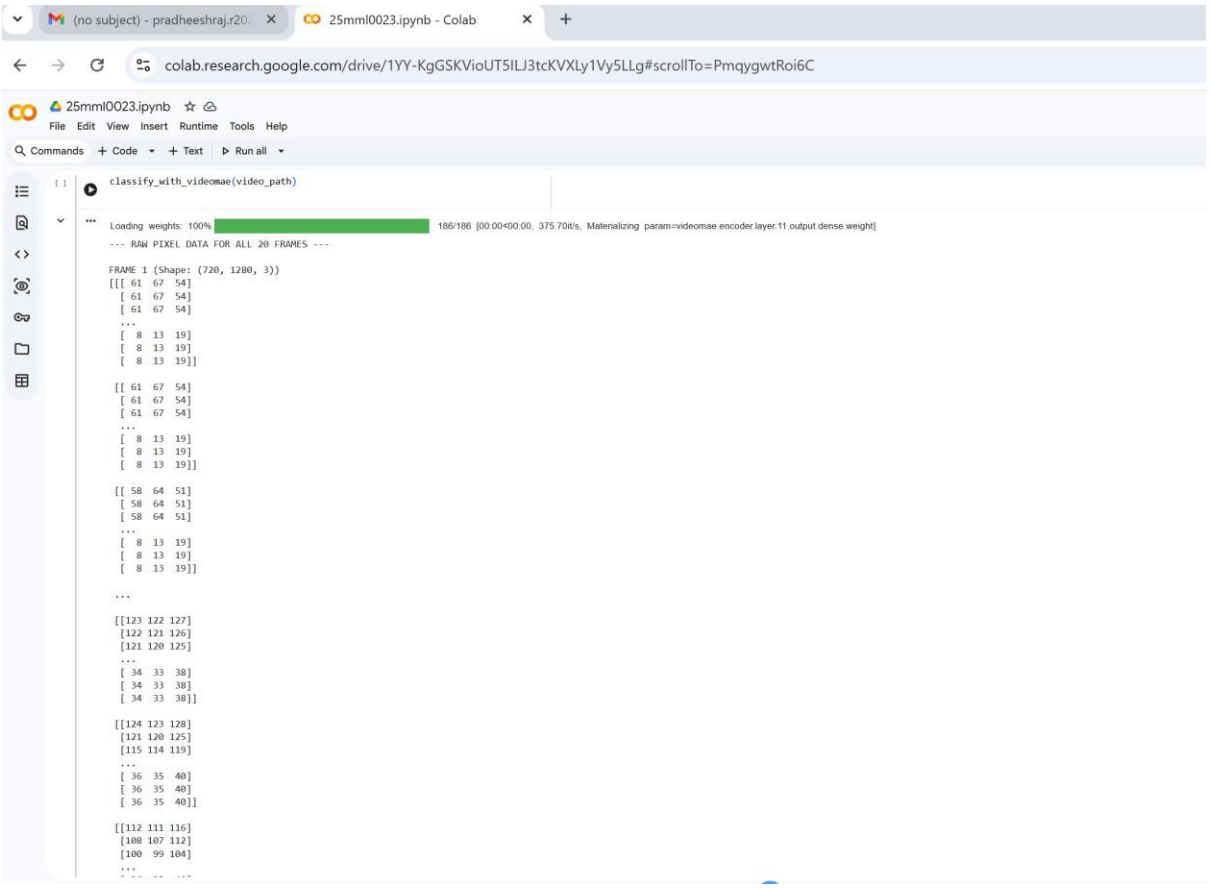
25mml0023.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all




```
--- Top 5 Predictions (Using 20-frame context) ---
pushing car: 18.87%
driving car: 17.09%
pushing cart: 5.15%
motorcycling: 3.08%
pushing wheelchair: 2.47%
```



CODE-2:

STEPS:

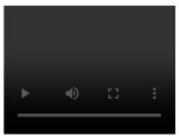
1. Import required libraries (cv2, numpy, matplotlib, subprocess, etc.).
2. Install OpenCV using pip install opencv-python (if not installed).
3. Import subprocess (optional, used for running system commands).
4. Provide the video file path.
5. Load the video using cv2.VideoCapture().
6. Check if the video opened successfully.
7. Read video metadata using cap.get() function.
8. Count total number of frames using cv2.CAP_PROP_FRAME_COUNT.
9. Get video frame width using cv2.CAP_PROP_FRAME_WIDTH.
10. Get video frame height using cv2.CAP_PROP_FRAME_HEIGHT.
11. Get frames per second (FPS) using cv2.CAP_PROP_FPS.

12. Read the first frame using cap.read().
13. Understand that OpenCV loads images in BGR format.
14. Convert BGR image to RGB using cv2.cvtColor() for matplotlib display.
15. Use modulo operator (frame_number % 100 == 0) to extract every 100th frame.
16. Display frames using matplotlib.pyplot.imshow().
17. Release video capture using cap.release().

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from glob import glob
import IPython.display as ipd
from tqdm.notebook import tqdm
```

```
#import subprocess
```

```
ipd.Video("026c7465-309f6d33.mp4",width = 200)
```



```
Start coding or generate with AI.
```

Open the Video and Read Metadata

```
!pip install opencv-python
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
```

```
# Load in video capture
import cv2
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
```

```
# Total number of frames in video
cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

```
2398.0
```

```
# Video height and width
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
print(f'Height {height}, Width {width}')
```

```
Height 720.0, Width 1280.0
```

```
# Get frames per second
fps = cap.get(cv2.CAP_PROP_FPS)
print(f'FPS : {fps:0.2f}')
```

```
FPS : 59.94
```

```
cap.release()
```

Pull the image from the first frame

```
cap = cv2.VideoCapture('026c7465-309f6d33.mp4')
ret, img = cap.read()                          #will return TRUE everytime we call this function till we reach the end of the video
print(f'Returned {ret} and img of shape {img.shape}')
```

```
Returned True and img of shape (720, 1280, 3)
```

Loading the First Image

```
plt.imshow(img)  #opencv loads color in BGR format, matplot lib expects RGB format
```

```
<matplotlib.image.AxesImage at 0x7faad2e85be0>
```

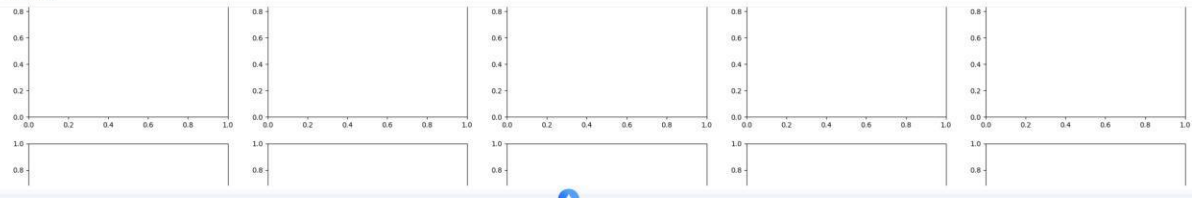

```
## Helper function for plotting opencv images in notebook
def display_cv2_img(img, figsize=(10, 10)):
    img_ = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=figsize)
    ax.imshow(img_)
    ax.axis("off")
```

```
display_cv2_img(img)
```

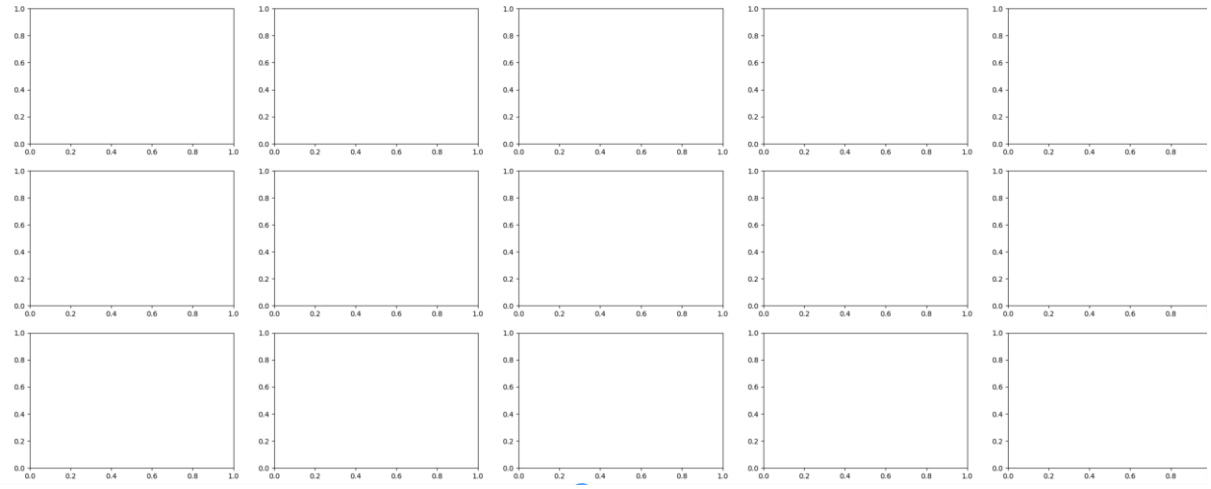
```
cap.release()
```

Display Multiple Frames from the Video

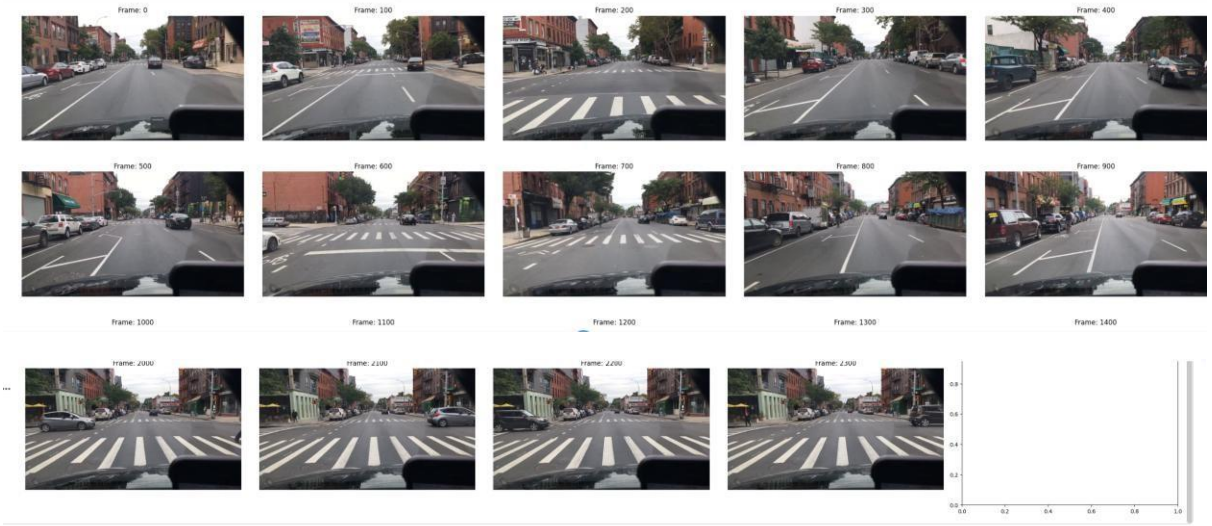
```
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
plt.show()
```



```
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()
plt.show()

# Without flatten → 2D grid (row, column)
# With flatten → 1D list (single index)
```



```
import cv2
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()

cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame % 100 == 0:                                    #frame is divisible by 100 - Note the modulo operator
        axs[img_idx].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axs[img_idx].set_title(f'Frame: {frame}')
        axs[img_idx].axis('off')
        img_idx += 1

plt.tight_layout()                                          #automatically adjusts the spacing between the plots
plt.show()
cap.release()
```




```
labels = pd.read_csv('mot_labels.csv',low_memory=False)
#mlabels.head()
#video_labels = (labels.query('videoName == "026c7465-309f6d33"').reset_index(drop=True).copy())
#video_labels["video_frame"] = (video_labels["frameIndex"] * 11.9).round().astype("int") #Video is at 60Hz but the labellers did only 5 Hz
```

```
#print(video_labels.shape)
```

```
(0, 14)
```

```
print(labels["category"].value_counts())
```

```
category
car              394391
pedestrian        67862
truck             22812
bus               10694
bicycle            3823
other vehicle      3756
rider              3307
motorcycle         2277
other person        782
trailer             376
train               156
Name: count, dtype: int64
```

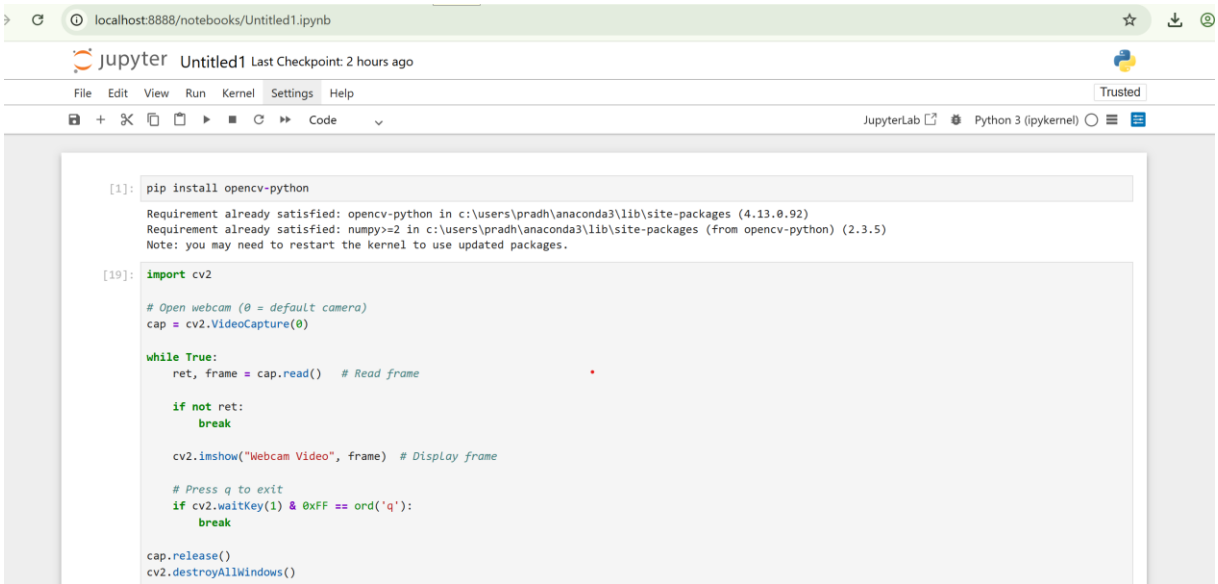
```
cap = cv2.VideoCapture("026c7465-309f6d33.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

img_idx = 0
for frame in range(n_frames):
    ret, img = cap.read()
    if ret == False:
        break
    if frame == 1035:
        break
```

```
cap.release()
```

```
display_cv2_img(img)
```

ON - DEVICE IMAGE RECOGNITION

CODE:



localhost:8888/notebooks/Untitled1.ipynb

jupyter Untitled1 Last Checkpoint: 2 hours ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[20]:
```
import cv2

cap = cv2.VideoCapture(0)
while True:
    ret, frame = cap.read()

    if not ret:
        break

    # Draw rectangle
    cv2.rectangle(frame, (100,100), (300,300), (0,255,0), 3)

    # Draw circle
    cv2.circle(frame, (200,200), 50, (255,0,0), 3)

    cv2.imshow("Shapes on Video", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

localhost:8888/notebooks/Untitled1.ipynb

jupyter Untitled1 Last Checkpoint: 2 hours ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python 3 (ipykernel)

[25]:
```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # Convert to grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Edge detection
    edges = cv2.Canny(gray, 100, 200)

    cv2.imshow("Original", frame)
    cv2.imshow("Canny Edge Detection", edges)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

localhost:8888/notebooks/Untitled1.ipynb

jupyter Untitled1 Last Checkpoint: 2 hours ago

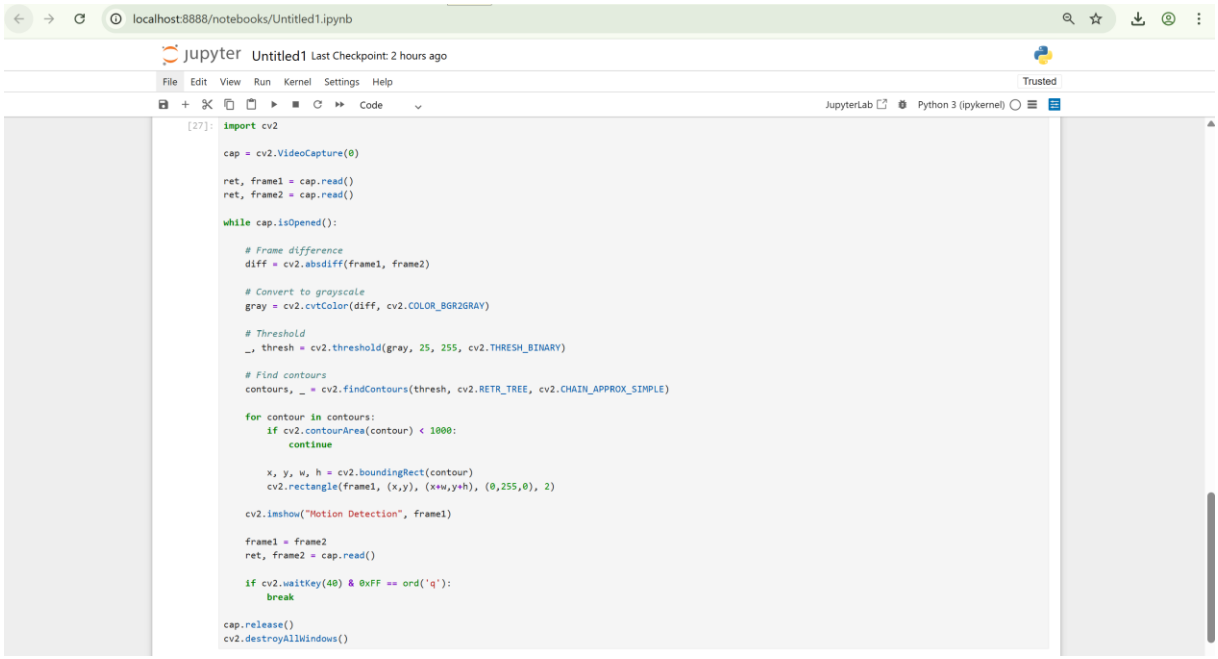
File Edit View Run Kernel Settings Help

Trusted

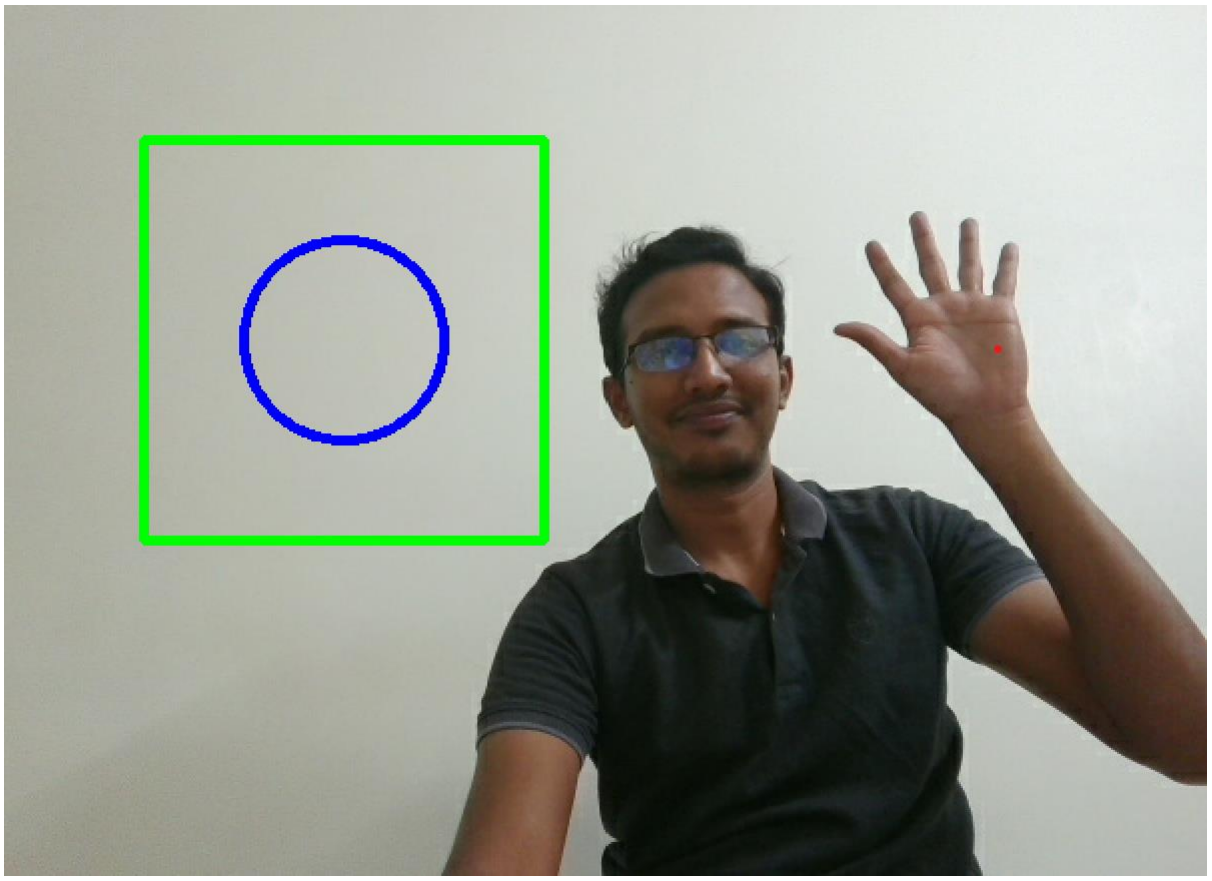
Code

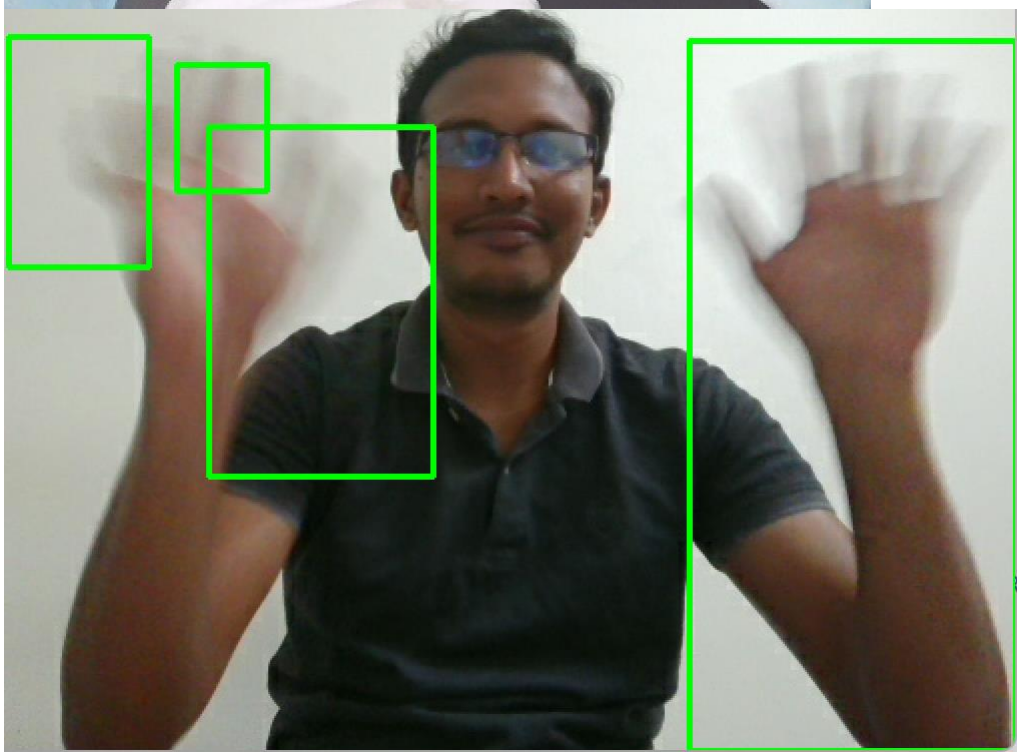
JupyterLab Python 3 (ipykernel)

[24]:
```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # Negative transformation
    negative = 255 - frame

    cv2.imshow("Original", frame)
    cv2.imshow("Negative Image", negative)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```


localhost:8888/notebooks/Untitled1.ipynb
jupyter Untitled1 Last Checkpoint: 2 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[27]:
import cv2
cap = cv2.VideoCapture(0)
ret, frame1 = cap.read()
ret, frame2 = cap.read()
while cap.isOpened():
# Frame difference
diff = cv2.absdiff(frame1, frame2)
# Convert to grayscale
gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)
# Threshold
_, thresh = cv2.threshold(gray, 25, 255, cv2.THRESH_BINARY)
Find contours
contours, _ = cv2.findContours(thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
for contour in contours:
if cv2.contourArea(contour) < 1000:
continue
x, y, w, h = cv2.boundingRect(contour)
cv2.rectangle(frame1, (x,y), (x+w,y+h), (0,255,0), 2)
cv2.imshow("Motion Detection", frame1)
frame1 = frame2
ret, frame2 = cap.read()
if cv2.waitKey(40) & 0xFF == ord('q'):
break
cap.release()
cv2.destroyAllWindows()

OUTPUT:





Explanation:

1. Webcam Video Capture

1. Import OpenCV library
   import cv2 is used to load the OpenCV module for image and video processing.
2. Initialize video capture object
   cv2.VideoCapture(0) is used to access the default webcam.
3. Start continuous loop
   A while True loop is used to capture video frames continuously.
4. Read frame from camera
   cap.read() captures a frame and returns the frame matrix.

5. Check frame status
   If the frame is not captured successfully, the program stops.
6. Display the video frame
   cv2.imshow() shows the captured frame in a window.
7. Control playback speed
   cv2.waitKey(1) waits for a short time between frames.
8. Check exit condition
   If the user presses the 'q' key, the loop stops.
9. Release camera resource
   cap.release() stops the webcam and releases system resources.
10. Close all OpenCV windows
    cv2.destroyAllWindows() closes all display windows.

2. Drawing Shapes (Rectangle & Circle)
   1. Import OpenCV library
      The cv2 module is imported for image processing.
   2. Open webcam video stream
      cv2.VideoCapture(0) is used to capture video from the camera.
   3. Start frame capture loop
      A loop is used to continuously read frames from the webcam.
   4. Capture video frame
      cap.read() reads the current frame from the camera.
   5. Draw rectangle on frame
      cv2.rectangle() draws a rectangle using coordinates.
   6. Define rectangle parameters
      The function requires starting point, ending point, color, and thickness.
   7. Draw circle on frame
      cv2.circle() draws a circular shape on the video frame.
   8. Specify circle properties
      Center point, radius, color, and thickness are given.
   9. Display modified frame
      cv2.imshow() shows the frame with shapes drawn on it.
   10. Exit and release resources
       Pressing 'q' exits the loop and closes the camera and windows.

3. Canny Edge Detection
   1. Import OpenCV library
      import cv2 loads the image processing library.
   2. Start video capture
      cv2.VideoCapture(0) opens the webcam.
   3. Capture video frames continuously
      A loop is used to process frames in real time.
   4. Read the frame from webcam
      cap.read() retrieves the current frame.
   5. Convert frame to grayscale
      cv2.cvtColor() converts the image from RGB to grayscale.
   6. Apply Canny edge detection
      cv2.Canny() detects edges based on intensity gradients.

7. Define threshold values
   Two threshold values control edge sensitivity.
8. Display original frame
   cv2.imshow() displays the original video.
9. Display detected edges
   Another window shows the edges detected in the frame.
10. Exit and release resources
    Pressing 'q' stops the program and closes all windows.

4. Negative Image Transformation
1. Import OpenCV library
   cv2 is used for image processing operations.
2. Start webcam capture
   cv2.VideoCapture(0) accesses the camera.
3. Begin frame capture loop
   A loop continuously reads frames.
4. Capture video frame
   cap.read() retrieves the frame.
5. Store frame matrix
   The frame is stored as a matrix of pixel values.
6. Apply negative transformation
   255 - frame inverts pixel intensity values.
7. Convert bright pixels to dark
   High intensity becomes low intensity.
8. Convert dark pixels to bright
   Low intensity becomes high intensity.
9. Display original and negative image
   Two windows show the original and negative frames.
10. Exit program properly
    Press 'q' to release the camera and close windows.

5. Motion Detection
1. Import OpenCV library
   cv2 is used for video analysis.
2. Open webcam stream
   cv2.VideoCapture(0) captures live video.
3. Capture two initial frames
   Two frames are needed to detect motion.
4. Calculate frame difference
   cv2.absdiff(frame1, frame2) finds pixel differences.
5. Convert difference to grayscale
   cv2.cvtColor() simplifies processing.
6. Apply thresholding
   cv2.threshold() converts the image to binary form.
7. Detect contours
   cv2.findContours() finds moving object boundaries.
8. Filter small movements
   Small contours are ignored using area threshold.

9. Draw bounding box around motion
   cv2.rectangle() highlights the moving object.
10. Update frames and display result
    The frame is updated continuously to track motion.

Mathematical Concepts Used in Image Processing:

1. Image Representation

A digital image is represented as a 2-D matrix of pixel intensity values.

Equation

$$I(x, y)$$

Explanation

- Irepresents the image.
- $x$and $y$represent pixel coordinates.
- Each element in the matrix stores the intensity value of a pixel.

Example matrix:

$$I = \begin{bmatrix} 120 & 135 & 150 \\ 98 & 110 & 130 \\ 75 & 90 & 105 \end{bmatrix}$$

Each number represents the brightness of that pixel.

2. Grayscale Conversion

Converts a color image (RGB) into a single intensity image.

Equation

$$Gray = 0.299R + 0.587G + 0.114B$$

Explanation

- R= Red intensity
- G= Green intensity
- B= Blue intensity

Green has the highest weight because the human eye is most sensitive to green light.

Example:

$$Gray = 0.299(100) + 0.587(150) + 0.114(200)$$
$$Gray = 29.9 + 88.05 + 22.8 = 140.75$$

So the grayscale pixel value ≈ 141.

3. Negative Image Transformation

Creates the inverse image by subtracting pixel values from the maximum intensity.

Equation

$$Negative = L - 1 - Pixel$$

Where:

- L= Maximum intensity level (256 for 8-bit images)

For 8-bit images:

$$Negative = 255 - Pixel$$

Explanation
Example:
Original pixel = 80

$$Negative = 255 - 80 = 175$$

This converts dark areas to bright and bright areas to dark.

4. Image Gradient (Edge Detection)
Used in edge detection algorithms like Canny.
Equation

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Gradient Magnitude

$$G = \sqrt{G_x^2 + G_y^2}$$

Explanation

- G_x= horizontal intensity change
- G_y= vertical intensity change
- High gradient value indicates edges in the image.

5. Frame Difference (Motion Detection)
Detects movement between two video frames.
Equation

$$Difference = | Frame_1 - Frame_2 |$$

Explanation

- If objects move, pixel values change.
- The absolute difference highlights moving areas.

Example:
Frame1 pixel = 120
Frame2 pixel = 150

$$| 120 - 150 | = 30$$

Large difference → motion detected.

6. Thresholding
Converts grayscale image into binary image.

Equation

$$g(x,y) = \begin{cases} 255 & \text{if } f(x,y) > T \\ 0 & \text{if } f(x,y) \leq T \end{cases}$$

Explanation

- T= threshold value
- Pixels above threshold → white
- Pixels below threshold → black

Used for object segmentation.

7. Contour Area Calculation

Used to detect objects in motion detection.

Equation

$$Area = \sum_{i=1}^{n} (x_i y_{i+1} - x_{i+1} y_i)$$

Explanation

- Used to calculate the area of detected contour boundaries.
- Small areas can be ignored to remove noise.

8. Bounding Rectangle

Used to draw rectangle around detected objects.

Equation

$$Rectangle = (x, y, w, h)$$

Where:

- x, y= starting coordinate
- w= width
- h= height

9. Pixel Intensity Range

For an 8-bit image:

Equation

$$0 \leq Pixel \leq 255$$

Explanation

- 0 → Black
- 255 → White
- Values in between → Shades of gray.

10. Edge Detection Using Sobel Operator

Used to detect horizontal and vertical edges.

Equation

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * I$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * I$$

Explanation
These matrices detect changes in pixel intensity, which represent edges.

NUMBER PLATE DETECTION

The Number Plate Recognition (NPR) project is a computer vision application designed to automatically detect and read vehicle license plate numbers from images. It is based on Automatic Number Plate Recognition (ANPR) technology, which combines image processing, object detection, and optical character recognition (OCR) to identify vehicles through their license plates. This technology is widely used in smart traffic management, automated parking systems, toll collection, and law enforcement.

System Overview

The system processes vehicle images and extracts the license plate number automatically. It uses Python-based computer vision libraries such as OpenCV and NumPy, along with machine learning models to analyze images and identify the license plate region.

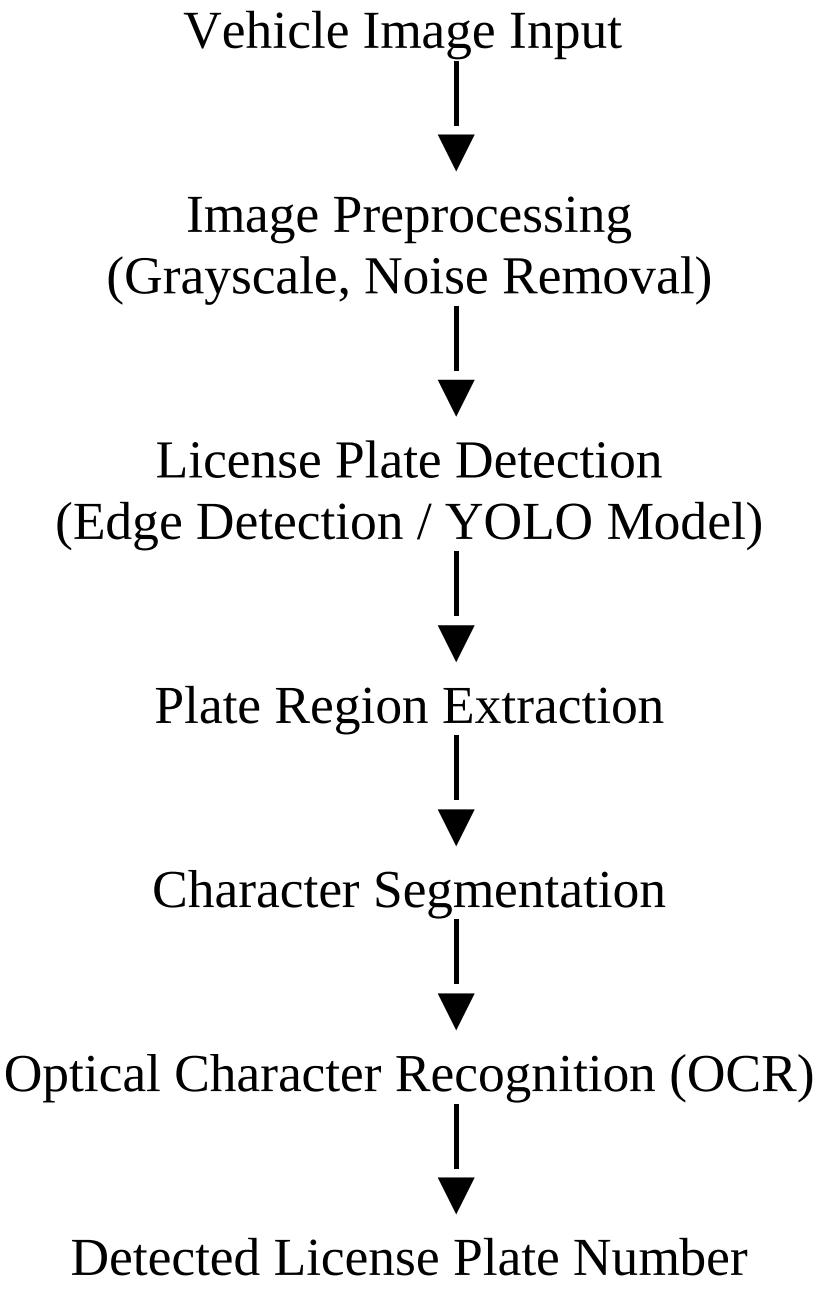
The process begins with image preprocessing, where the input image is converted to grayscale and filtered to remove noise. These steps improve the image quality and highlight edges, which helps the system detect important features.

Next, the system performs number plate detection. In this stage, computer vision techniques such as edge detection, contour detection, or object detection models like YOLO are used to identify the region containing the license plate. Once the plate area is located, the system extracts the plate region from the image.

After detecting the plate region, the system performs character segmentation, where each letter and number on the license plate is separated individually. Image processing techniques such as thresholding and contour analysis are used to isolate characters from the background.

The final step is character recognition using Optical Character Recognition (OCR). Deep learning models such as Convolutional Neural Networks (CNN) or OCR engines like Tesseract convert the segmented characters into readable text. The recognized characters are then combined to form the final vehicle number.

Architecture Diagram – Image Processing Pipeline

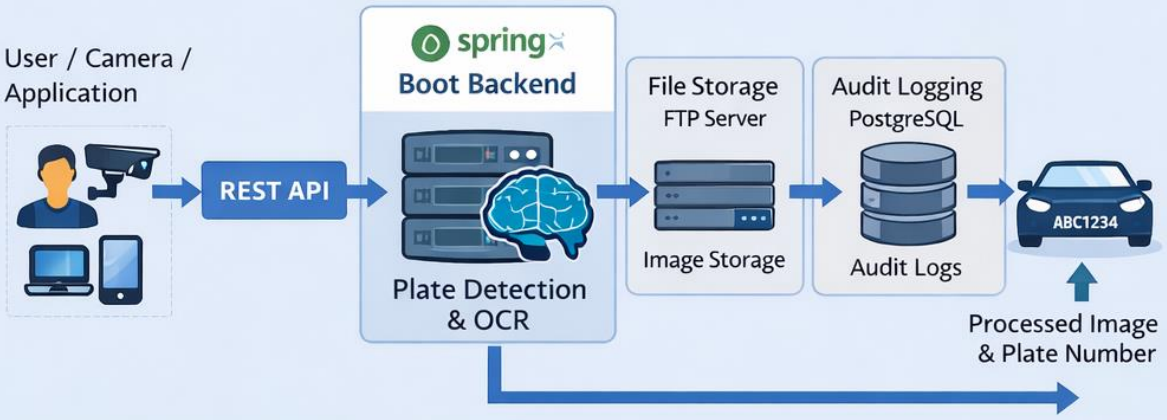


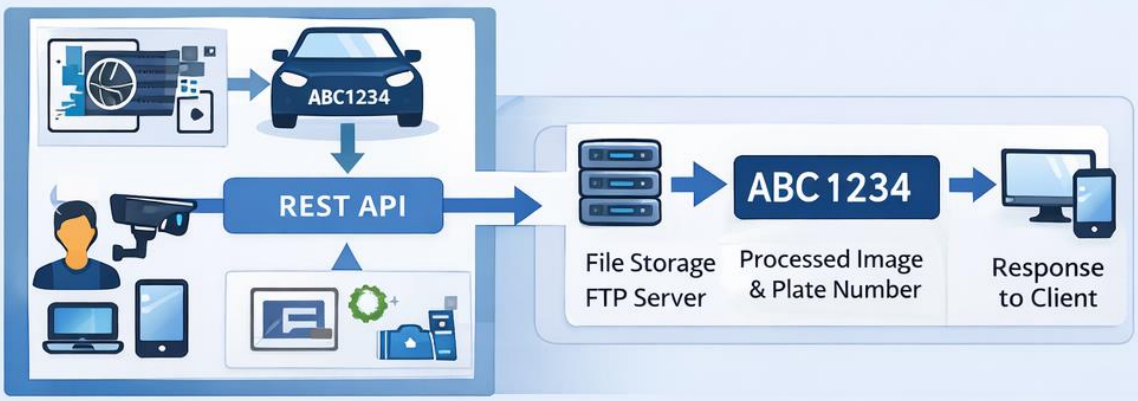
Number Plate Recognition System

Image Processing Pipeline

Vehicle Image Input → Image Preprocessing (Grayscale, Noise Removal) → License Plate Detection (Edge Detection / YOLO) → Plate Region Extraction → Character Segmentation → Detected License Plate Number

Backend System Architecture





Backend System Architecture

The system also includes a backend microservice responsible for managing image processing requests, storing files, and maintaining system logs. The backend is implemented using Spring Boot (Java) and exposes REST APIs to interact with the application.

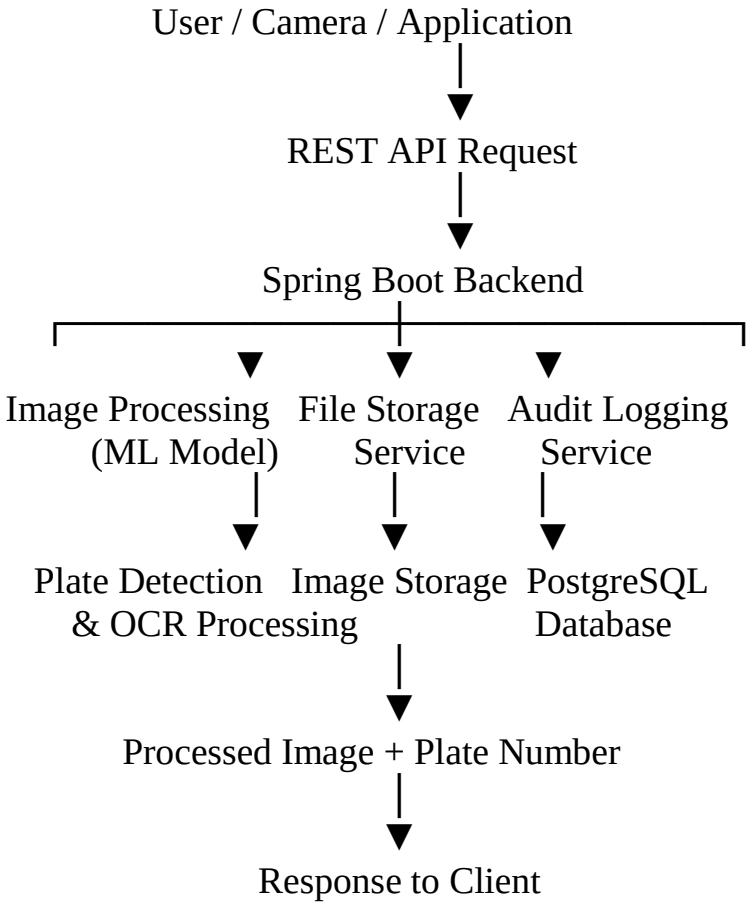
The architecture follows a multi-layered design, separating the application into controllers, service logic, and data access layers. This improves maintainability, scalability, and modularity.

The backend interacts with:

- A machine learning model for number plate detection
- A remote storage server for image storage
- A PostgreSQL database for maintaining audit records

Spring Data JPA and Hibernate are used for database communication, allowing the application to store and retrieve data using Java objects instead of raw SQL queries.

Architecture Diagram – Complete System



Core Backend Functionalities

The backend provides several important functionalities:

Image Upload and Processing

Users can upload vehicle images to the backend service. The system processes the image using the machine learning model and generates an output image containing detected number plates with bounding boxes and labels.

Image Retrieval

Stored images can be retrieved by the client. The backend streams the image data directly to the client and sets the correct image format for display.

Bulk Data Download

The system also supports downloading multiple files together by compressing them into a single archive before sending them to the client.

Database Auditing

To ensure traceability and accountability, the system records every file upload operation in a database. Each record contains details such as the file name and the timestamp of the upload. This auditing mechanism provides a reliable history of system activity and helps monitor how the system is used.

Challenges and Solutions

One of the main challenges during development involved FTP directory navigation and authentication issues. The problem occurred when uploading files to deeply nested directories. This issue was solved by resetting the FTP client's working directory before creating the required structure. Additionally, authentication problems were resolved by configuring appropriate user permissions on the server.

Conclusion

The Number Plate Recognition system demonstrates how computer vision, machine learning, and backend microservices can work together to automate vehicle identification. The project integrates image processing, object detection, OCR, and cloud-based file management into a single scalable solution. When deployed with CCTV cameras or traffic monitoring systems, it can automatically detect and record vehicle numbers without human intervention, making it highly useful for smart city infrastructure and intelligent transportation systems.